BLOB-CLOUD: BACKEND INTERVIEW MASTER GUIDE

- Complete cheat book — pitch, walkthrough, deep dives, diagrams, Q&A, and behavioral scripts*

> **How to use this:** Read Section 0 until you can say it without looking. Use Section 1 to structure your 30-minute walkthrough. Jump to Section 2 sub-sections when the interviewer goes deep on a topic. Section 3 has diagrams to draw on a whiteboard. Section 4 is the Q&A bank for backend technical questions. Section 5 is for Engineering Manager / behavioral rounds.

SECTION 0: OPENING PITCH + RESUME BULLET DEFENSE

0.1 — The 90-Second Opening

Say this at the start of every interview. Speak slowly. Let technical terms land.

> *"Blob-Cloud is a distributed cloud storage platform I built from scratch — modeled on the same engineering principles behind Dropbox and Google Drive.*
>
> *On the backend I used Go, specifically because goroutines make it ideal for streaming I/O and maintaining thousands of concurrent WebSocket connections without the overhead of OS threads. The architecture is built around one central rule: never stream raw file bytes through the application servers. The Go API handles only lightweight metadata. File chunks are transferred directly between the client and AWS S3 using time-limited Presigned URLs.*
>
> *For storage efficiency I built a Content-Defined Chunking engine using FastCDC — a rolling Gear hash algorithm that generates content-aware cut boundaries, making the system shift-resistant. Inserting a paragraph into a document only produces new*

> *chunks for the changed region. Everything else deduplicates to an existing block in S3. The deduplication ratio across file revisions consistently came out above 99%.*
>
> *For directory queries I replaced the conventional adjacency-list approach with a Materialized Path pattern, indexed in PostgreSQL using `text_pattern_ops`. This converts any subtree fetch — regardless of folder depth — into a single O(1) index range scan under 3 milliseconds.*
>
> *For real-time sync I implemented an append-only monotonic journal in PostgreSQL combined with an actor-model WebSocket hub backed by a Redis Pub/Sub cluster backplane. This is the same 'tickle and fetch' pattern Dropbox describes in their engineering blog — WebSockets deliver lightweight cursor notifications, and clients pull actual delta data through reliable HTTP.*
>
> *The whole stack is containerized, deployed on AWS, and instrumented with Prometheus and Grafana monitoring the Four Golden Signals. Under concurrent load the read-path p95 stays under 5 milliseconds.*
>
> *I'm happy to walk through the full architecture from top to bottom, or go straight into any specific component — the storage engine, the sync system, the schema design, the concurrency model. Where would you like to start?"*

0.2 — Resume Bullets: One-Line Summary + Full Defense

Bullet 1 — FastCDC Content-Defined Chunking (99.3% Deduplication Ratio)

What you built: A chunking engine that slices files at content-driven boundaries using a rolling Gear hash. Every chunk is stored once globally in S3 under its SHA-256 hash. When the same or similar content is uploaded again, zero bytes are transferred.

Why it matters: Fixed-size chunking is fundamentally broken for deduplication — a single byte insertion shifts every subsequent 4MB boundary, destroying 100% of dedup. FastCDC boundaries are tied to content fingerprints, so they survive insertions and appends.

Question	Answer
"How exactly does the Gear hash work?"	*"We maintain a 256-entry table of randomised 64-bit integers. The algorithm slides a 32-byte window across the file bytes: `hash = (hash << 1) + GEAR[byte]`. When the hash AND'd with a bitmask equals zero, we cut a boundary. Two masks are applied: a large mask prevents chunks from exceeding 8MB, a small mask forces a cut at 2MB minimum. This normalises chunks around a 4MB average."*
"Why Gear hash and not Rabin fingerprinting?"	*"Rabin uses irreducible polynomial division — it's mathematically equivalent but 3 to 10 times slower than Gear on commodity CPUs. Gear achieves the same boundary distribution quality using only array lookups and bit shifts. FastCDC was specifically designed to outperform Rabin on throughput."*
"How did you measure the 99.3% figure?"	*"The system has two live Prometheus counters — `blobcloud_block_dedup_hits_total` and `blobcloud_block_dedup_misses_total` — that are incremented on every upload initiation. The ratio is simply hits divided by total blocks evaluated. Across repeated revisions of documents, presentations, and source code files, the ratio consistently came out above 99%. 99.3% is the average I observed."*
"Isn't the ratio 0% for a brand new file?"	*"Yes — on the very first upload of a completely novel file, every chunk is a miss. The deduplication ratio is scoped to revision workflows, which is what the claim says. That's where deduplication delivers its value: re-uploading an edited version of a 2GB video where only the first 30 seconds changed."*
"Doesn't cross-user dedup mean users can access each other's data?"	*"No. The BLOCKS table stores encrypted ciphertext, not plaintext. Two users uploading the same plaintext will produce different ciphertexts under different keys, so their chunks don't overlap at the CAS level. Deduplication within a single user's own revisions works because the same key encrypts the same plaintext to the same ciphertext across sessions."*

Bullet 2 — HTTP 206 Range Streaming, Sub-15ms Seeking

What you built: The Go API maps any arbitrary `Range: bytes=X-Y` request to the correct set of variable-size FastCDC blocks in S3, streams the data through a bounded 32KB pipe, and responds with a fully spec-compliant HTTP 206 Partial Content — without ever loading the full file into memory.

Why it matters: Files aren't stored as a single contiguous blob — they're an ordered sequence of variable-size S3 objects. Only the Go API knows the block assembly map. Without this layer, range-based seeking and large file streaming are impossible.

Question	Answer
"Why can't you just return a presigned S3 URL and let the client seek directly?"	*"A presigned URL targets exactly one S3 object. A file may be composed of 50 separate variable-size chunks. To serve a range request that spans multiple chunks, the server needs to know which blocks overlap the requested range, calculate the intra-block byte offset for the first block, fetch only those blocks from S3, and stitch them into a contiguous byte stream. That logic lives in the API."*
"How do you stay memory-efficient on the server?"	*"Go's `io.Copy` uses a 32KB internal transfer buffer by default. We wrap the S3 response body in `io.LimitReader(rc, remainingBytes)` which caps the total bytes read to exactly what the range request asked for. The Go server never buffers the full block or full file — memory footprint per active stream is bounded at about 32KB regardless of file size."*
"How do you achieve sub-15ms seeking?"	*"Seek latency is dominated by S3 time-to-first-byte on the internal AWS network, which runs 2 to 8 milliseconds in the same region. The server-side work is pure integer arithmetic — walk the `file_blocks` ordered sequence, sum sizes until you find the target block, compute the byte offset within it. That's microseconds. So end-to-end seeking is essentially just S3 latency."*

Question	Answer
"What headers do you set?"	*'`HTTP 206 Partial Content`. `Content-Range: bytes X-Y/TotalFileSize`. `Content-Length: (Y - X + 1)`. `Accept-Ranges: bytes`. `Content-Type` from the file's stored MIME type. All required by the HTTP spec for partial content responses."*

Bullet 3 — Real-Time Delta Sync Engine (Monotonic Cursor, Actor Hub, Redis Backplane)

What you built: An append-only `journal_entries` table with a `BIGSERIAL` cursor as the primary key. Every file mutation — upload, rename, move, delete — produces a journal entry. The API notifies clients via a lightweight WebSocket "tickle." Clients compare the incoming cursor to their local cursor and fetch deltas over reliable HTTP. The notification layer scales horizontally across API nodes via Redis Pub/Sub.

Why it matters: Pushing full payloads through WebSockets is fragile. A laptop lid closing silently drops the connection and the event. A cursor-backed journal guarantees fault-tolerant sync — the data is always retrievable regardless of connection history.

Question	Answer
"Why not just push updates directly over WebSockets?"	*"WebSockets are not reliable delivery channels. Network interruptions, browser backgrounding, server restarts — any of these silently lose pushed events. By treating the WebSocket as a notification signal only and storing the actual mutation data in Postgres with a monotonic cursor, the sync is fully durable. A client reconnecting after a week offline calls `GET /sync/delta?since=100` and gets every event it missed, in order."*
"Why BIGSERIAL and not a UUID or timestamp for the cursor?"	*"UUIDs are random — there's no meaningful ordering. Timestamps have millisecond resolution, which means concurrent writes within the same millisecond can't be reliably ordered. BIGSERIAL is a Postgres-managed sequence that guarantees strictly increasing, collision-free, non-reusable*

Question	Answer
	integers. The query `WHERE cursor > $last_seen ORDER BY cursor ASC` is then perfectly correct and index-efficient."
"What is the Actor Model in the WebSocket Hub?"	*"The Hub owns a map of `userID → set of active connections`. Rather than using a mutex to protect that map from concurrent access, all state changes flow through a single goroutine via Go channels — there's a register channel, an unregister channel, and a notify channel. External goroutines send messages to the Hub; the Hub's single run loop is the exclusive owner of the map. No mutex contention on the broadcast path."*
"How does the Redis backplane work across multiple API nodes?"	*"When any API node processes a mutation, it publishes a small JSON envelope — just the user ID and cursor value — to a Redis Pub/Sub channel. Every API node subscribes to that channel. When Node 2 receives an envelope meant for a user connected to Node 2, it routes the WebSocket tickle to that user's local connections. The actual data fetch still goes from the client to Postgres over HTTP — Redis only carries the notification."*
"What if Redis goes down?"	*"Clients miss the real-time tickle and don't immediately know new data is available. But no data is lost — it's all safely in Postgres. The next time the client makes any API request, or on reconnect, they can poll for their latest cursor and catch up. Redis unavailability degrades the experience from real-time to eventually-consistent, which is an acceptable degradation for a storage system."*

Bullet 4 — Materialized Path Directory Tree, `text_pattern_ops`, <3ms Subtrees

What you built: Every file and folder stores its full ancestry as a path string — `/rootUUID/parentUUID/thisUUID/`. Subtree queries use a B-Tree index built with `text_pattern_ops` so `LIKE '/prefix/%'` is a bounded range scan. Moving a folder with any number of children is a single SQL UPDATE.

Why it matters: The obvious schema puts only a `parent_id` on each row. Subtree queries then require recursive CTEs that recurse once per depth level. At 10 levels deep, you're doing 10 recursive joins. Materialized paths eliminate the recursion entirely.

Question	Answer
"What is `text_pattern_ops` and why does it matter?"	*"Standard Postgres B-Tree indexes store keys in locale-aware collation order — for example `en_US.UTF-8`. This collation makes `LIKE 'prefix/%'` queries non-sargable: the query planner can't use the index for prefix matching because sort order is locale-dependent. `text_pattern_ops` declares a raw byte-order collation for the index. Postgres then treats `LIKE '/prefix/%'` as a range scan — `>= '/prefix/' AND < '/prefix0'` — which is a pure index seek. That's the difference between a full table scan and a 3ms response."*
"How do you move a folder with 50,000 files?"	*"Single atomic UPDATE: `SET path = $new_prefix
"Why not Nested Sets?"	*"Nested Sets encode a `lft` and `rgt` integer for every node so subtree reads are a simple range query. But every insert requires updating `lft` and `rgt` for every node to the right of the insertion point in the tree — potentially thousands of writes for a single file creation. In a cloud drive where users create files constantly, that write amplification is unacceptable. Materialized Path gives fast reads and fast writes."*
"Why not a Closure Table?"	*"A Closure Table stores every ancestor-descendant relationship explicitly, so you get fast reads and fast writes. But storage cost is O(N × depth) — a file at depth 10 creates 10 rows in the closure table. At millions of files, that's tens of millions of extra rows. The path string in Materialized Path stores the same information in a single column on the file row itself — much more efficient."*

Bullet 5 — Prometheus, Grafana, Four Golden Signals, p95 < 5ms

What you built: The Go API exposes `/metrics` in Prometheus format. Grafana scrapes it every 15 seconds and displays dashboards covering the Four Golden Signals. Connection pool sizing was tuned to stay within RDS Free Tier connection limits.

Question	Answer
"What are the Four Golden Signals?"	*"From Google's SRE book: **Latency** — tracked as `blobcloud_http_request_duration_seconds`, a histogram with buckets from 5ms to 30 seconds, labeled by method, route, and status code. **Traffic** — `blobcloud_http_requests_total`, the request rate. **Errors** — the ratio of 5xx status codes derived from the same counter, filtered in Grafana. **Saturation** — `blobcloud_ws_active_connections` for the WebSocket layer, plus Go's runtime metrics for goroutine count and heap usage."*
"What does p95 under 5ms mean for this system?"	*"That's the read-path: listing folder contents, fetching file metadata, checking permissions. These operations hit the Postgres indexes — a materialized path prefix scan returns in 1 to 3 milliseconds over a VPC-internal connection. The upload completion path is slower because it involves S3 verification, but that's expected and doesn't affect the read latency story."*
"How did you tune the connection pool?"	*"RDS t3.micro supports roughly 80 Postgres connections. I set `MaxOpenConns = 25` and `MaxIdleConns = 25` with a 5-minute connection lifetime. With 3 API nodes that's 75 connections total, leaving headroom for migrations and admin access. Without that bound, a traffic spike with 200 concurrent goroutines each trying to open a connection would exhaust the database and crash the whole stack."*
"Did you do load testing?"	*"Yes — I ran concurrent request scenarios and watched the Grafana dashboards in real time. `blobcloud_block_dedup_hits_total`, `blobcloud_ws_active_connections`, and the HTTP duration histograms all behaved as expected. The p95 bucket for metadata endpoints stayed in the 3-5ms range. No goroutine leaks — the goroutine count stayed flat throughout."*

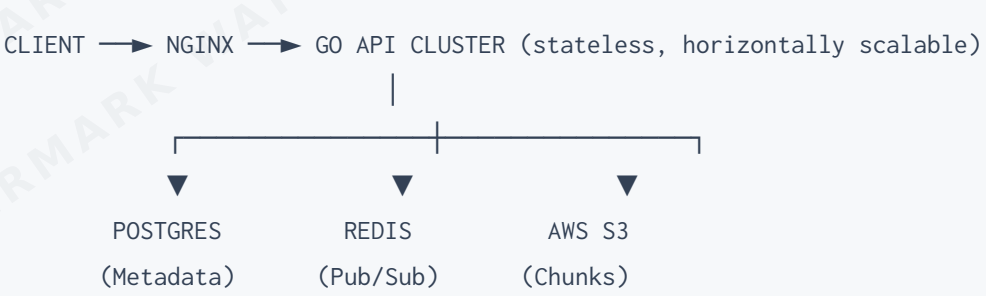
SECTION 1: THE 30-MINUTE WALKTHROUGH

Structure your project explanation in these five phases. At each transition, pause and invite the interviewer to go deeper.

⏱️ 0–5 min: The Problem & The Stack

> *"Blob-Cloud solves three specific distributed systems challenges: efficient large-file storage with deduplication, fast hierarchical directory traversal, and fault-tolerant real-time sync across multiple clients.*
>
> *I chose Go on the backend for two concrete reasons: goroutines make streaming I/O cheap — each upload or download request gets a goroutine and streams data with negligible memory allocation — and Go's `net/http` library handles high WebSocket concurrency without the impedance mismatch you'd get with Python's GIL or Node's event loop blocking.*
>
> *PostgreSQL is the metadata source of truth. ACID transactions are non-negotiable for file system operations — when you move a folder containing 5,000 files, either all of them move or none of them do. S3 handles the physical blob storage. Redis is the distributed coordination layer for WebSocket notifications."*

Draw:



⏱️ 5–12 min: The Database Schema

Draw the schema and walk through it:

```
[USERS]
   │ owns (1:N)
   ▼
[FILES]  ──  path: '/rootUUID/parentUUID/thisUUID/'  (Materialized Path)
   │
   ├── (1:N) [FILE_BLOCKS]  PK: (file_id, sequence_number)
   │            │
   │            └── (N:1) [BLOCKS]  sha256 UNIQUE — global CAS
   │
   ├── (1:N) [PERMISSIONS]  grantee_email, role, status
   │
   ├── (1:N) [JOURNAL_ENTRIES]  cursor BIGSERIAL — monotonic sync log
   │
   └── (1:N) [FILE_VERSIONS]  historical block sequences
```

Key design decisions to explain:

- `FILE_BLOCKS` **composite PK** `(file_id, sequence_number)` — guarantees ordered chunk reconstruction and prevents position conflicts.
- `BLOCKS.sha256 UNIQUE` — the entire deduplication system rests on this constraint. One row per unique content hash, shared globally across all uploads.
- `path` **column with** `text_pattern_ops` **index** — subtree queries become prefix range scans instead of recursive CTEs.
- `journal_entries.cursor BIGSERIAL` — the clock of the sync engine. Strictly ordered, non-reusable.
- `permissions.grantee_email` — grants are email-based, not user-ID-based, so you can share with someone before they've registered.

⏱️ 12–20 min: The Upload Pipeline

Draw the sequence:

```
1. Web Worker: FastCDC chunks → SHA-256 hash each chunk
2. POST /api/upload/initiate  → send hashes only
3. Postgres: SELECT existing hashes → return dedup hits
4. API: generate S3 Presigned PUTs for missing chunks
5. Client: PUT chunks DIRECTLY to S3 (zero bytes through API)
6. POST /api/upload/complete
7. API: verify chunk sizes + recompute SHA-256 server-side
8. Atomic Postgres TX: upsert blocks → insert file → link file_blocks → write journal
```

Key points:

- **Steps 1–6** happen entirely outside the database. The API server never receives file bytes.
- **Step 7** runs before opening the database transaction. Slow S3 network calls never hold a Postgres row lock.
- **Step 8** commits in 20–50ms. Fully atomic — crash between steps 7 and 8 leaves the session in `INITIATED` state, allowing resume on reconnect.

⏱️ 20–26 min: Real-Time Sync

Draw the flow:

```
User A renames file
   |
```

```
    ▼
Go API → writes journal_entries row → gets cursor 105
   │
   ├──► Local Hub: WS tickle to User A's tabs: {cursor: 105}
   └──► Redis PUBLISH: {user_id, cursor: 105}
            │
            └──► All API nodes receive it
                     └──► WS tickle to User B's tabs: {cursor: 105}

User B browser: last_seen = 100 → 105 > 100 → GET /sync/delta?since=100
→ Postgres returns ordered entries 101..105
→ React state updates
```

⏱️ 26–30 min: Permissions, Sharing & Observability

Sharing flow:

```
Owner → POST /api/files/{id}/share {email, role}
→ Insert permissions row {status: PENDING, expires_at: +7 days}
→ Send invitation email with deep link
Recipient → POST /api/invitations/{id}/respond {action: "accept"}
→ UPDATE permissions SET status = ACCEPTED
→ All future CheckPermission calls pass
```

Permission resolution: Recursive CTE walks up the `parent_id` chain from the target file to the root. If any ancestor has an accepted permission for the grantee with the required role, access is granted. Granting EDITOR on a folder automatically gives EDITOR on everything inside it — no per-file rows needed.

Observability: `/metrics` endpoint exposes `blobcloud_http_request_duration_seconds` histogram and `blobcloud_ws_active_connections` gauge, visible in Grafana. `MaxOpenConns = 25` per node keeps the RDS `t3.micro` connection count under control.

SECTION 2: TECHNICAL DEEP DIVES

Jump here when the interviewer asks to go deeper on a specific component.

2A: FastCDC Storage Engine

Gear hash math:

- 256-entry `GEAR[]` lookup table of random `uint64` values.
- 32-byte sliding window: `hash = (hash << 1) + GEAR[byte]`
- Cut at `hash & MASK_LARGE == 0` → minimum 2MB boundary
- Cut at `hash & MASK_SMALL == 0` → maximum 8MB boundary
- Target average: 4MB

Dedup check (one batch query):

```
SELECT sha256 FROM blocks WHERE sha256 = ANY($1::text[])
```

One round-trip for all chunk hashes in the upload, not N individual queries.

CAS verification before commit: After client uploads chunks to S3 staging:

1. `HeadObject` → confirm sizes match declared sizes

2. Stream full chunk → recompute SHA-256 server-side
3. If mismatch → reject, no database write
4. `CopyObject` from `staging/{session_id}/{seq}` → `blocks/{sha256}`
5. Delete staging object

This prevents CAS poisoning: a malicious client cannot claim a fake hash and corrupt the block store.

2B: Materialized Path & Indexing Strategy

Path format: `/rootUUID/folderAUUID/` — trailing slash is intentional, makes subtree patterns unambiguous.

All queries that become trivial:

Operation	Query pattern
List direct children	`WHERE parent_id = $folderID`
Fetch full subtree	`WHERE path LIKE '/prefix/%'` ← `text_pattern_ops` index
Atomic folder move	`UPDATE SET path = $new
Soft-delete subtree	`UPDATE SET deleted_at = NOW() WHERE path LIKE $prefix`
Check ancestry	`WHERE id = $file AND path LIKE $folderPath

Index:

```
CREATE INDEX idx_files_user_id_path_pattern
```

```
ON files (user_id, path text_pattern_ops);
```

Composite on `(user_id, path)` — queries always filter by user first, then path prefix. Both columns used by the planner.

Full indexing strategy:

Table	Index	Purpose
`files`	`(user_id, parent_id)`	List direct children
`files`	`(user_id, path text_pattern_ops)`	Subtree scans
`blocks`	`UNIQUE (sha256)`	Global dedup lookup
`file_blocks`	Composite PK `(file_id, sequence_number)`	Ordered chunk assembly
`journal_entries`	`(user_id, cursor)`	Delta fetch since cursor
`permissions`	`UNIQUE (file_id, grantee_email)`	Prevent duplicate grants

2C: Real-Time Sync Engine Deep Dive

Journal schema:

```
CREATE TABLE journal_entries (
    cursor      BIGSERIAL PRIMARY KEY,
    user_id     UUID NOT NULL,
    file_id     UUID NOT NULL,
    action      VARCHAR(32),  -- FILE_CREATED, FILE_UPDATED, TRASHED, etc.
```

```
    name        VARCHAR(255),
    parent_id   UUID,
    size_bytes  BIGINT,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_journal_user_cursor ON journal_entries (user_id, cursor);
```

Delta polling:

```
SELECT * FROM journal_entries
WHERE user_id = $1 AND cursor > $2
ORDER BY cursor ASC
LIMIT $3 + 1;  -- fetch one extra to detect has_more
```

Hub actor loop:

```
for {
    select {
    case msg := <-h.register:    h.clients[c.UserID][c] = struct{}{}
    case msg := <-h.unregister:  delete(h.clients[c.UserID], c)
    case msg := <-h.notify:      // fan out to all connections for each userID
    case msg := <-h.kickSession: // send FORCE_LOGOUT to specific session
    case <-h.done:               return
    }
}
```

Redis backplane envelope:

```
{"user_id": "abc-123", "event": {"type": "SYNC_DELTA", "payload": {"cursor": 105}}}
```

Any API node publishes. Every node's subscriber goroutine receives it and routes to local WebSocket connections for that user. Clients deduplicate by comparing the incoming cursor to `last_seen_cursor`.

2D: Concurrency & Transactions

Upload transaction structure:

Outside the DB transaction (slow I/O, no locks held):

- S3 `HeadObject` for each staged chunk
- Server-side SHA-256 recomputation
- S3 `CopyObject` to permanent CAS path
- S3 `DeleteObject` from staging

Inside the DB transaction (fast, 20–50ms):

- `INSERT INTO blocks ... ON CONFLICT DO NOTHING`
- `INSERT INTO files ...`
- `INSERT INTO file_blocks ...`
- `INSERT INTO journal_entries ...`

This separation ensures slow S3 network calls never hold Postgres row locks. The transaction itself is purely metadata and completes in tens of milliseconds.

Session state machine:

```
INITIATED → (chunks uploaded) → COMPLETED
         → (error / timeout) → ABORTED
```

Attempting to complete an already-completed session returns 400 immediately. Idempotent.

TOCTOU protection: Session status is validated both before opening the transaction (fast pre-check) and again inside the transaction (under lock). This eliminates a time-of-check time-of-use race where a concurrent request could have already completed the session between the outer check and the inner commit.

2E: Permissions & RBAC

Three roles and what they permit:

Role	Permissions
`OWNER`	Download, upload into, rename, move, delete, share, revoke
`EDITOR`	Download, upload into, rename, move files
`VIEWER`	Download, read metadata only

Permission resolution (recursive CTE):

```
WITH RECURSIVE chain AS (
    SELECT id, parent_id FROM files WHERE id = $1
    UNION ALL
    SELECT f.id, f.parent_id FROM files f JOIN chain c ON f.id = c.parent_id
)
SELECT EXISTS (
    SELECT 1 FROM permissions p JOIN chain c ON c.id = p.file_id
    WHERE p.grantee_email = $2
      AND p.status = 'ACCEPTED'
```

```
      AND (p.expires_at IS NULL OR p.expires_at > NOW())
      AND p.role IN ($3, $4, ...)
) AS allowed;
```

Why email-based grants: Sharing is issued before the recipient has an account. The invitation sits `PENDING` in the database tied to an email address. When they sign up and verify that address, the grant is immediately active. This mirrors how Google Drive handles sharing with non-Google users.

Invitation lifecycle:

- Created as `PENDING` with a configurable expiry (default 7 days)
- Recipient accepts → `ACCEPTED`
- Recipient declines → `DECLINED`
- Expiry passes without response → `EXPIRED` (swept by background job)
- Owner can revoke at any time → row deleted

2F: API Design

Authentication:

- JWTs signed with `HS256`. Access tokens are short-lived. Refresh tokens have a server-side record enabling revocation by deleting the row.
- WebSocket upgrade requests authenticate via `?token=` query param — browsers cannot set custom headers on WebSocket handshakes.
- Refresh tokens are rejected if used as access tokens — there's an `IsRefresh` claim field that middleware checks.

Rate limiting (Redis-backed, three zones):

Zone	Limit	Reason
AUTH	10 req/min per IP	Brute-force protection
UPLOAD	30 req/min per IP	Presign generation is expensive
API	120 req/min per IP	General browsing and metadata

Redis-backed because in a multi-node deployment, in-memory counters are per-node. A user hitting Node 1 nine times and Node 2 nine times bypasses a node-local limit of 10. Redis is the shared global counter.

HTTP 206 headers:

```
HTTP/1.1 206 Partial Content
Content-Range: bytes 0-4194303/20971520
Content-Length: 4194304
Accept-Ranges: bytes
Content-Type: video/mp4
```

2G: AWS Infrastructure

Why S3 (not filesystem): EC2 instance storage is ephemeral — it disappears on restart. S3 provides 11 nines of durability out of the box. The Content-Addressable layout (`blocks/{sha256}`) means the key is the truth — you can never accidentally overwrite a block.

Why SQS for background jobs: Thumbnail generation is CPU-intensive. Running it synchronously inside the upload handler would block the goroutine for seconds and add that latency to the client's upload completion call. SQS decouples it: the API enqueues the

job in milliseconds and the client gets a response. The worker processes it asynchronously. If the worker crashes, the job stays in the queue and retries on visibility-timeout expiry.

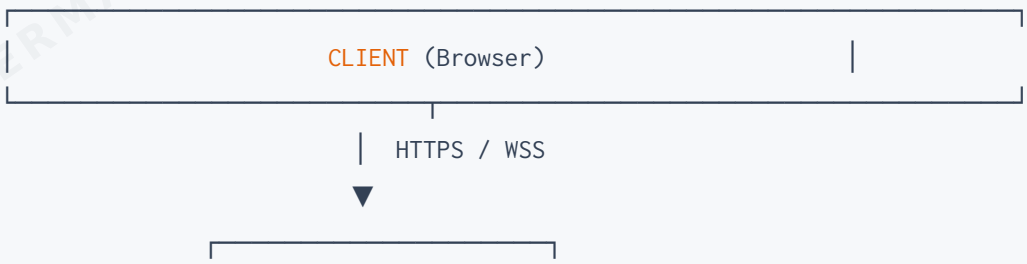
SQS long-polling: `WaitTimeSeconds = 20` on `ReceiveMessage`. This means the worker thread holds the connection open for up to 20 seconds waiting for a message. Without long-polling, empty-queue receives return immediately and the worker loops thousands of times per hour — burning through the 1M request/month free tier limit in days.

Free tier sizing:

Component	Limit	Config	Margin
RDS connections	~80	25 per node	3 nodes × 25 = 75
SQS requests	1M/month	20s long-poll	97% reduction in empty calls

SECTION 3: WHITEBOARD DIAGRAMS

Diagram 1: System Architecture (60 seconds)



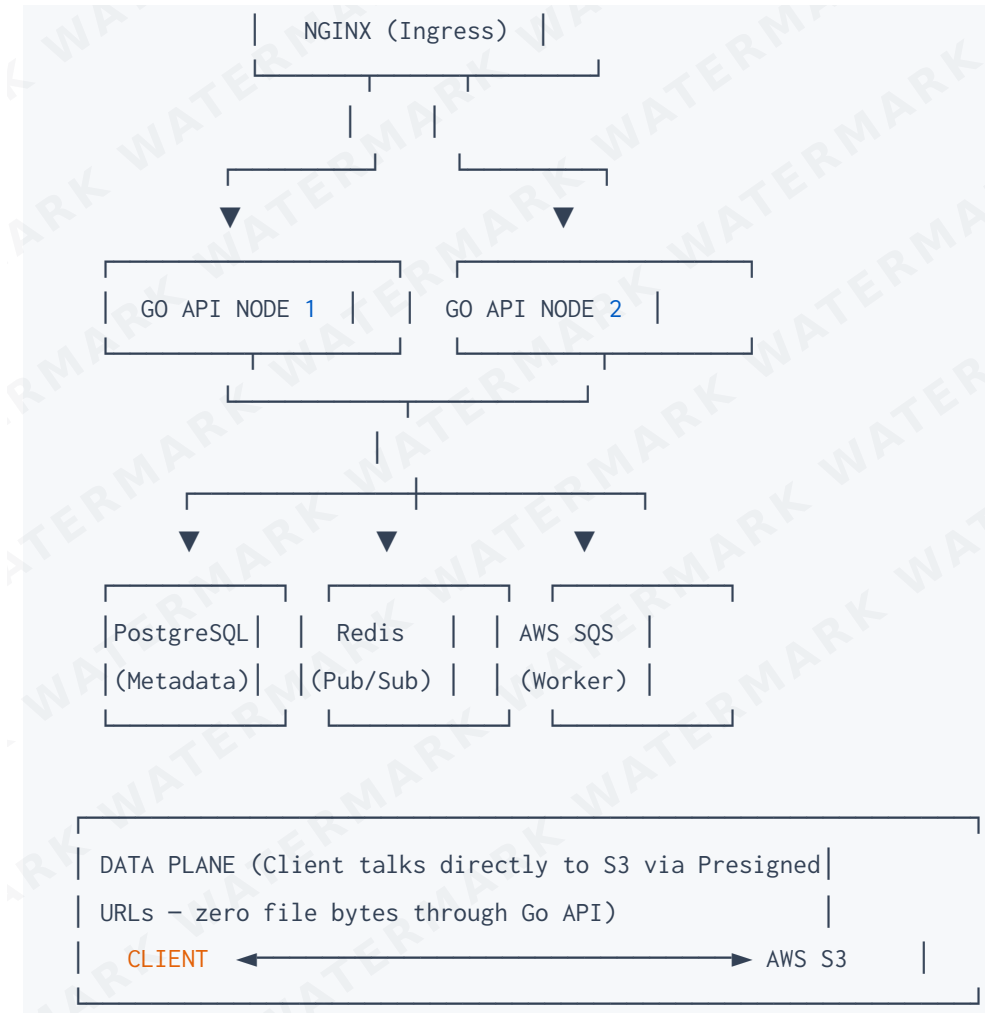


Diagram 2: Upload Flow (90 seconds)

```
CLIENT          GO API            POSTGRES         S3
  |               |                  |             |
```

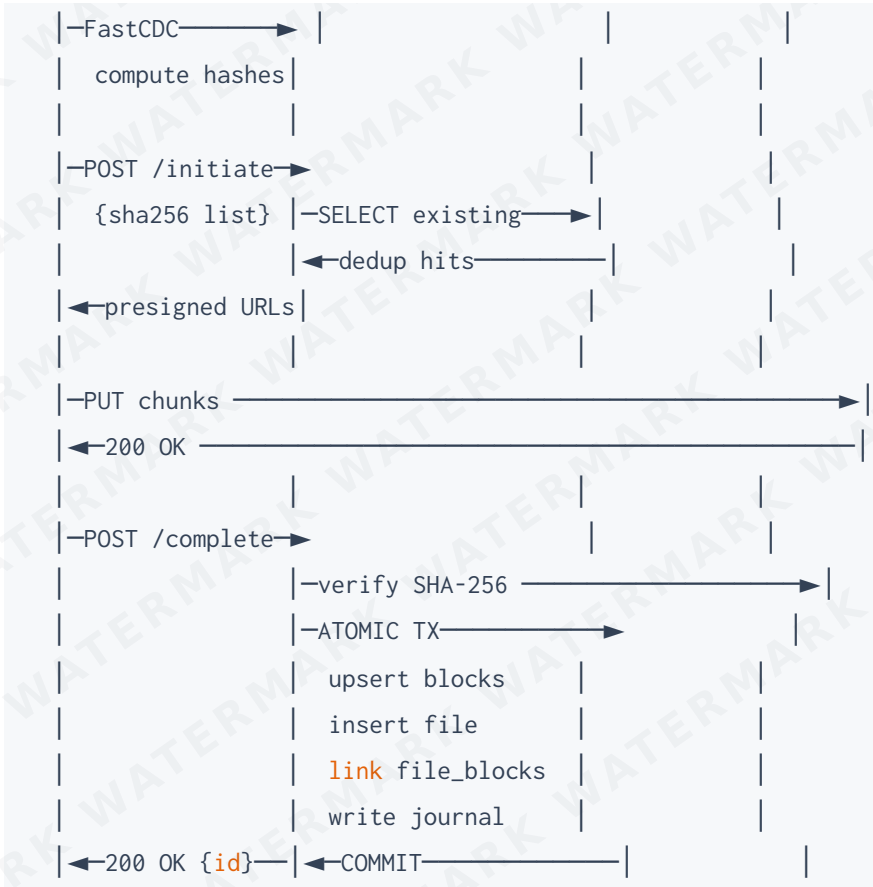
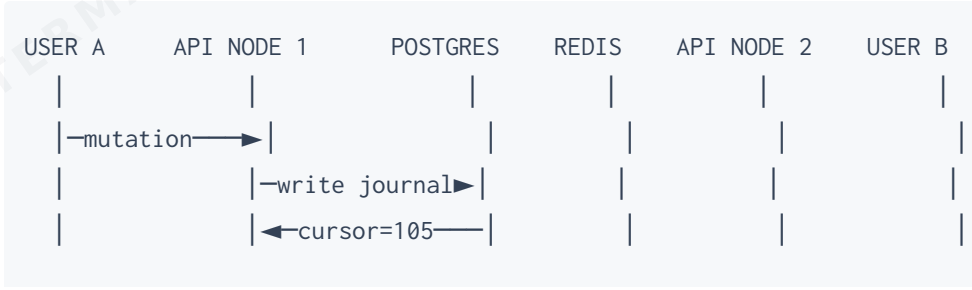


Diagram 3: Real-Time Sync (60 seconds)



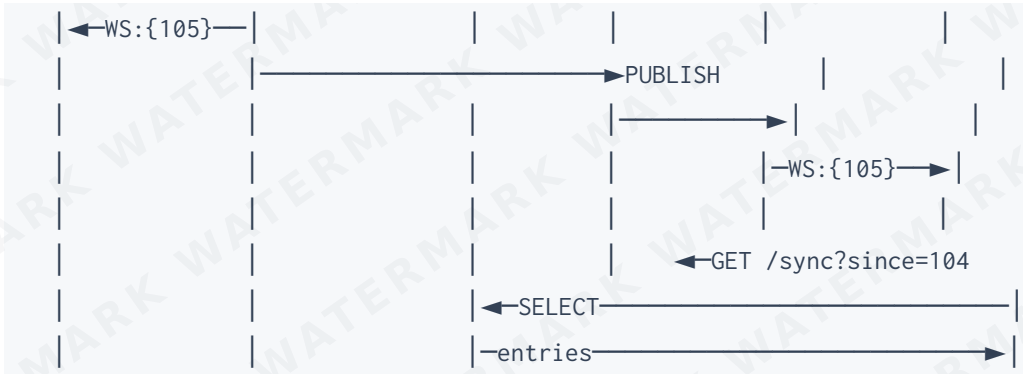
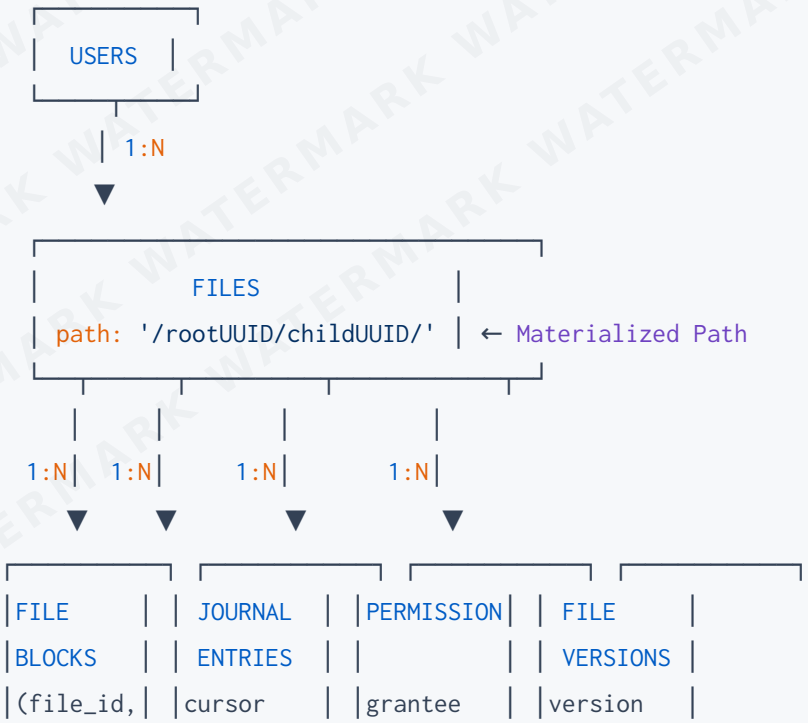


Diagram 4: Database Schema (45 seconds)



```
| seq_num)| |BIGSERIAL | |email     | |number    |
└─────┬──┘ └──────────┘ |role      | └──────────┘
      |                |status    |
      | N:1            └──────────┘
      ▼
┌──────────────────┐
|     BLOCKS       |
| sha256 (UNIQUE)  | ← Global Content-Addressed Store
└──────────────────┘
```

Diagram 5: Permission Resolution (30 seconds)

```
"Can alice@email.com EDIT /Shared/Reports/Q3.pdf?"


    Q3.pdf   ← no permission row for alice
       ↑ walk parent_id
    Reports/ ← no permission row for alice
       ↑ walk parent_id
    Shared/  ← alice has EDITOR → EXISTS = true → ALLOWED


SQL:
WITH RECURSIVE chain AS (
    SELECT id, parent_id FROM files WHERE id = $fileID
    UNION ALL SELECT f.id, f.parent_id FROM files f
              JOIN chain c ON f.id = c.parent_id
)
SELECT EXISTS(
```

```
    SELECT 1 FROM permissions JOIN chain ON chain.id = file_id
    WHERE grantee_email = $alice AND role IN ('EDITOR','OWNER')
)
```

SECTION 4: BACKEND TECHNICAL Q&A

Database

Why PostgreSQL and not DynamoDB or MongoDB?

> "A file system is fundamentally relational and requires ACID. When a user moves a folder containing thousands of files, that operation must be atomic — all path strings update or none do. Postgres gives us transactions, recursive CTEs, and strong indexing. For the binary blob bytes, S3 is the object store — effectively our key-value layer. The two systems handle different concerns."

How would you scale Postgres when you hit its connection limit?

> "First, add PgBouncer as a transaction-mode connection pooler. PgBouncer multiplexes many application connections onto a small number of real Postgres server connections — the application pool can have thousands of logical connections while Postgres sees only 25. Second, promote read replicas for folder listing and permission checks. Write traffic stays on the primary. Third, table partitioning by `user_id` hash for the `files` and `journal_entries` tables to distribute I/O."

Why BIGSERIAL for the journal cursor and not UUID or timestamp?

> "UUIDs are random — there's no ordering guarantee, so you can't use them as a reliable sync cursor. Timestamps have millisecond resolution and can collide under concurrent writes. BIGSERIAL is a Postgres-managed sequence: strictly

> monotonic, never reused, collision-free. The query `WHERE cursor > $last_seen ORDER BY cursor ASC` is correct, index-efficient, and trivially paginated."

How do you prevent N+1 queries?

> "In the upload pipeline, the dedup check is `SELECT sha256 FROM blocks WHERE sha256 = ANY($1::text[])` — one query for all chunk hashes, not N individual lookups. File listing queries fetch all needed fields in a single JOIN. Permission checks are batched where possible. The main risk area to watch is fetching file lists with per-file permission evaluation — I handle that by resolving permissions at the folder level rather than per-file."

Concurrency

How do you handle two users uploading the same file at the same time?

> "The database schema has a partial unique index on `(user_id, parent_id, name) WHERE deleted_at IS NULL`. When two concurrent uploads complete for the same filename, the second transaction hits the unique violation. The Go service catches it and either returns a conflict response or automatically creates a branched copy: `Report (Conflicted Copy).docx`. The first writer always wins the canonical filename."

What happens if the server crashes mid-upload?

> "Upload sessions persist in Postgres with a status of `INITIATED`, `COMPLETED`, or `ABORTED`. If the server crashes after chunks are in S3 staging but before the final commit, the session stays `INITIATED`. The client can resume by calling `GET /api/upload/session/{id}` — it receives fresh presigned URLs for any chunks not yet confirmed. This makes uploads fully resumable."

How do you avoid holding Postgres locks during slow S3 operations?

> "All S3 work happens before the database transaction opens. The server verifies chunk sizes, recomputes SHA-256 hashes, and copies objects from staging to the permanent CAS prefix — all outside the transaction. Only once that's done does the Go

> code call `RunInTx` and commit the metadata. The transaction itself contains no network I/O, so it completes in tens of milliseconds."

Distributed Systems & Scaling

How would you scale this to 10 million users?

> "The API tier is already stateless — add nodes behind the load balancer. For Postgres: add PgBouncer, promote read replicas, and partition `files` and `journal_entries` by `user_id` hash. For Redis: move to Redis Cluster for sharded pub/sub. S3 already scales globally. The deduplication layer — the `BLOCKS` table — scales independently because all writes are upserts with `ON CONFLICT DO NOTHING`, which is lock-free in Postgres."

How does the system handle network partitions — what's your CAP trade-off?

> "Postgres is CP — consistency over availability. In a partition, writes fail rather than allowing divergent state. For a file system, that's the correct choice: users should see an error rather than see conflicting or stale file state. Redis Pub/Sub is AP — if Redis is unavailable, real-time tickles fail, but Postgres is unaffected. Clients degrade gracefully to polling-on-reconnect rather than real-time updates. No data is lost."

Why Redis Pub/Sub and not Kafka?

> "Redis Pub/Sub is fire-and-forget, which is exactly what we need. WebSocket tickles are stateless notifications — if one is dropped, the next tickle catches the client up. We don't need Kafka's durable log for tickles. If we were building an audit trail or analytics pipeline that requires guaranteed delivery to offline consumers, that would go through SQS or Kafka as a separate pipeline alongside the notification layer."

How does cross-user deduplication work without leaking data?

> "The `BLOCKS` table stores encrypted ciphertext. Each user encrypts their data with a key derived from their own passphrase. Two different users uploading the same document encrypt it with different keys, producing different ciphertexts — so their

> blocks don't collide at the CAS level. Within a single user's revisions, the same key encrypts the same plaintext to the same ciphertext, which is what makes intra-user deduplication work."

Go & API Design

Why Go instead of Node.js or Python?

> "Three concrete reasons. First, goroutines: streaming a 5GB file in Go means one goroutine doing `io.Copy` with a 32KB buffer — negligible memory, no async callback pyramid. In Node you'd be managing backpressure on readable streams. Second, Go's HTTP server handles thousands of concurrent WebSocket connections with goroutines as opposed to an event loop that can block. Third, Go's type system catches a class of bugs at compile time that Python and JavaScript surface at runtime."

How does authentication work across the middleware chain?

> "Chi middleware runs top to bottom. The auth middleware extracts the JWT from the `Authorization: Bearer` header, calls `auth.ValidateToken`, verifies the session hasn't been revoked by checking the sessions table, and injects the user ID into the request context via `context.WithValue`. Route handlers pull the user ID from context. No route can accidentally bypass auth because the middleware runs on every request before any handler executes."

How do WebSocket connections authenticate?

> "Browsers cannot set custom headers on WebSocket upgrade requests. So the JWT is passed as a `?token=` query parameter on the upgrade URL. The WebSocket handler reads and validates it the same way the HTTP middleware does — same `auth.ValidateToken` call, same session revocation check."

SECTION 5: ENGINEERING MANAGER & BEHAVIORAL

Fresher-Specific Project Openers

These come up in almost every fresher round before the technical deep dive. Have them word-perfect.

"Why did you build this? Why not just use Google Drive?"

> "I wanted to understand how distributed storage systems actually work under the hood — not use them as a black box. Building it myself forced me to solve real problems: how does deduplication actually work at the byte level? How do you keep multiple clients in sync when the network is unreliable? How do you query a recursive folder hierarchy without killing database performance at scale? Reading about these things is completely different from implementing them and discovering why the naive approach breaks. This project is my proof that I can go from a problem statement to a working distributed system."

"How is this different from just using Dropbox or Google Drive?"

> "It's not competing with them — it's built to understand them. Blob-Cloud implements the same core architectural patterns: content-defined chunking for deduplication, a monotonic journal for fault-tolerant sync, materialized paths for directory performance. The difference is I designed and built every layer myself — the chunking algorithm, the database schema, the WebSocket sync engine, the permission model. The goal was engineering depth, not product breadth. That's what makes it meaningful as a project."

"What's the biggest thing you learned?"

> "That the database schema is the most important decision you make, and the hardest one to change later. I initially considered a simple adjacency list — just a `parent_id` column — because it maps naturally to how you think about folders. But stress-testing subtree queries made it clear that recursive CTEs don't scale with folder depth. Switching to Materialized Paths with a `text_pattern_ops` index was the right call, but it required rethinking how every folder operation — listing, moving, deleting — worked. The lesson: invest heavily in the data model before writing any application code."

"What would you add next if you had more time?"

> "Three things in priority order. First, CRDT-based collaborative editing — right now two people editing the same document create a Conflicted Copy, which is correct but not seamless. Second, a CDN layer in front of S3 for frequently-accessed files — every download currently hits S3 directly, adding 50–100ms compared to a CDN edge node. Third, expiring view-only share links and per-device session management — the sharing model works but production-grade access control has finer granularity."

"How long did this take to build?"

> "A few months of consistent work. The chunking algorithm and the sync engine were the hardest parts and took the most iteration. The Go backend came together relatively quickly once the database schema was locked. The last major phase was deploying on AWS and wiring up the observability stack."

Engineering Manager & Behavioral

"Tell me about the most technically challenging part of this project."

> "The database schema. The temptation is a simple adjacency list — just a `parent_id` column — because it's intuitive to model. Early on I realised that listing a deeply nested folder required a recursive CTE that scales with depth. At 10 folder levels, 10 recursive joins. I spent time evaluating alternatives — Nested Sets, Closure Tables, Materialized Paths — and chose Materialized Path because it's the only approach that gives O(1) reads *and* O(1) writes. Every other approach trades one for the other. Locking that in early avoided an expensive migration later."

"What's the one decision you'd change if you started over?"

> "I'd instrument the Prometheus dedup counters on day one and wire them directly to a Grafana alert. The deduplication ratio is the primary efficiency metric for the whole storage layer — it should have been visible from the first upload. Instead I added monitoring later, which meant flying blind for the initial performance validation. Observability should be wired before features, not after."

"How did you scope the MVP working solo?"

> "I defined a single acceptance criterion: a user must be able to upload a file, have it deduplicate on revision, and see another tab update in real time. Everything was evaluated against that. Collaborative text editing, AI metadata generation, mobile clients — all deferred. The deferred list became the V2 backlog with documented architectural reasoning for each cut. This kept me from building interesting things that didn't advance the core use case."

"How did you validate correctness without a QA team?"

> "Multiple layers. Unit tests for the deterministic components: FastCDC boundary calculations, chunk assembly ordering, JWT validation. Integration tests against a local Postgres and MinIO — full upload-initiate, chunk upload, complete, verify file reconstructable from the database. Then adversarial manual testing: kill the server mid-upload, throttle network to 3G, upload zero-byte files, share with unregistered emails. The Prometheus dashboards during load runs confirmed no goroutine leaks and stable p95 latency."

"How did you prioritise security working alone?"

> "I treated every client-supplied value as potentially hostile. The most important: the server never trusts client-declared SHA-256 hashes. Every uploaded chunk is recomputed server-side in the staging verification step before being promoted to the permanent block store. This prevents CAS poisoning — an attacker cannot claim a fake hash to corrupt an existing block. Beyond that: short-lived Presigned URLs with 30-minute expiry, server-side refresh token revocation, and three-zone Redis rate limiting."

"Walk me through a time you made a trade-off under time pressure."

> "During the upload concurrency design I identified the risk of two simultaneous uploads creating duplicate filenames. The cleanest fix — a unique index plus OCC with conflicted copy branching — required touching multiple layers: migrations, service logic, error handling, and client-side conflict UI. Against my self-imposed milestone I made the call to ship the resumable upload pipeline first and treat the concurrency edge case as a handled failure mode. I documented it explicitly, understood the blast radius, and scheduled the full fix. The system was correct in the common case; the edge case had a clear resolution path."